

Advanced R Notes:

TOC

Chapter 1	Page 1	Chapter 9	Page 22	Chapter 17	Page 31
Chapter 2*	Page 2	Chapter 10	Page 25	Chapter 18*	Page 33
Chapter 3	Page 6	Chapter 11	Page 27	Chapter 19*	Page 41
Chapter 4	Page 8	Chapter 12	Page 28	Chapter 20	Page 46
Chapter 5	Page 10	Chapter 13	Page 29	Chapter 21	Page 48
Chapter 6	Page 12	Chapter 14	NA	Chapter 22*	Page 49
Chapter 7	Page 16	Chapter 15	NA		
Chapter 8*	Page 18	Chapter 16	NA		

*

= more indepth notes on chapters I struggled with conceptually or felt I needed to understand better

- Chapter 2
- Chapter 8
- Chapter 18
- Chapter 19
- Chapter 22

Chapter 1(good to know):

Suggested ways to develop packages: devtools, roxygen, testthat and usethis

Meta-techniques:

1. Reading Source Code: Will help you write better code, help develop your opinions about good and bad code
2. Adopting a Scientific Mindset: useful when you don't know how something works

Chapter 2: Names and Values

-> Difference between an object and its name

Quiz:(was very lost will return post chapter)

2.2 Binding Basics:

```
x <- c(1, 2, 3)
```

R is creating an object(the vector of values) and binding it to the name x.

So: Value doesn't have a name, the name is assigned a value.

Thus: Name serves as a reference to the value

Doing `y <- x` means y refers to the og object

Further: both x and y have the same location identifier(we can see when we do `lobstr::obj_addr()`, from `library(lobstr)`)

Syntactic-name

- must consist of letters, digits, . and, _ but can't start with _ or a digit
- You cannot use a **reserved word**:
 - Ex: TRUE, NULL, if, function
 - Full list: @ ? Reserved
- Name that doesn't follow these conditions is **non-syntactic**
 - **Can override non-syntactic names with backticks**
 - Eg: ``_abc``
 - Don't use quotes to override bad names, its outdated

2.2.2 Exercises:

- 1) a is R creating a new object, and the name a is assigned to it, b and c are then assigned to the same object. d is R creating a new object, with a different address(but the object is equivalent to a, b, and c)
- 2) Come back to 2
- 3) Automatically converting non-syntactic to
- 4) Look into `make.names`

2.3 Copy-on-modify

In R, when two variables point to the same object, modifications to one variable may not affect the other. This is due to **copy-on-modify** behavior.

For example:

```
x <- c(1, 2, 3)
y <- x
```

```
y[[3]] <- 4
```

```
x
```

```
#> [1] 1 2 3
```

Immutable: Where R objects are unchangeable (not the best term so don't really use)

#Note: RStudio treats c-o-m a bit differently

2.3.1 tracemem()

- Useful function in that it tells us an object's current address, and then whenever it is copied states the new address
- You can turn it off using untracemem()

2.3.2 Function Calls

Note the below:

```
f <- function(a) { a }
```

```
x <- c(1, 2, 3)
```

```
cat(tracemem(x), "\n")
```

```
#> <0x7fe1121693a8>
```

```
z <- f(x)
```

```
# no copy occurs
```

Inside the function, a and x both point to the same object, so no copy is made.

The value is not copied because the function doesn't modify the object.

2.3.3 Lists

Lists use shallow copies (only the list structure is copied), but the objects the list elements point to are not copied.

For Example:

```
l1 <- list(1, 2, 3)
```

```
l2 <- l1
```

```
l2[[3]] <- 4
```

Modifying an element in l2 creates a copy of l2 with the changed value, leaving l1 unchanged.

2.3.4 Data frames

Dataframes: lists of vectors

- When you modify a column, only that column needs to be modified, others will point to original references
- When you modify a row every column is modified which means every column must be copied

2.3.5 Character Vectors

- Where R uses references
- Drawn like `x <- c("a", "a", "abc", "d")`
- **Global string pool** where each element is a pointer to unique string in pool

2.4 Object Size

- `lobstr::obj_size()`: How much memory an object takes
- Elements of a list are refs to vals, size maybe much smaller than expected
- Global string pool makes it so that char vecs take up much less space
 - Repeating a string 100 times doesn't mean it takes up 100 times as much memory
- Thus combining size of objects is the size of the union of those objects
- Alt Rep: when we store certain vector very compactly: every seq no matter how large is the same size

2.5 Modify -in-place:

- Modifying typically creates a copy, but two exceptions"
 - **2.5.1 Single binding get special optimization**
 - Sometimes will still make copies when it doesn't need to
 - Better to determine empirically with `tracemem()`
 - **2.5.2 Environments are always modified in place**
 - When you modify an env all existing binding in an env continue to have to same ref

2.6 Unbinding and the garbage collector

- Garbage collector runs automatically when R needs more memory
- Near impossible to tell when it is going to run
 - Call `gcinfo(TRUE)` to print a message every time it is going to run
 - You can force it to run but never any need to call `gc()`
- `lobstr::mem_used()`: prints the total number of bytes used

Chapter 2: Names and Values

1. What are the rules for object names, and what happens if you try to use a not-allowed name with `<-` or with `make.names()`?

Rules for Naming:

- Object names must start with a letter or a dot (.) not immediately followed by a number.
- Names can contain letters, numbers, underscores (`_`), and dots.

- Names cannot match reserved words in R (e.g., if, else, TRUE, FALSE).

Using Not-Allowed Names:

- If you try to assign to a disallowed name using `<-`, R will throw an error, e.g., Error: unexpected input.
- If you use `make.names()`, R modifies the name to make it valid by replacing invalid characters with dots or appending a number to avoid duplication.

2. What does *copy-on-modify* mean and why is it important for designing fast/efficient code?

Copy-on-Modify:

- R uses a lazy copying mechanism to optimize memory usage. When you assign an object to a new variable (e.g., `y <- x`), both variables point to the same object in memory.
- A copy is only made if you modify `x` or `y`. This is called "copy-on-modify."

Importance:

- Efficient memory usage
- Performance optimization:
 - Avoids excessive memory allocation and speeds up computation

3. Why might the size of a *list* seem surprisingly small, even if it contains large objects?

The size of a list may seem small even if it contains large objects because:

- Lists store references (pointers) to the objects they contain, not the objects themselves.
- The memory footprint of the list only accounts for the overhead of maintaining these references, not the actual size of the objects.

Chapter 3: Vectors

1. What is the relationship between vectors, lists, matrices, and data frames?

Vectors:

- The simplest R data structure, containing elements of the same type (e.g., numeric, character, logical).
- One-dimensional.

Lists:

- A flexible data structure that can contain elements of different types, including vectors, other lists, and complex objects.
- Lists are hierarchical; they can hold multiple dimensions by containing sublists.

Matrices:

- A two-dimensional structure, where all elements must be of the same type.
- Essentially a vector with a dim attribute specifying rows and columns.

Data Frames:

- A two-dimensional structure where each column is a vector, and columns can have different types (like a list of vectors).
- Essentially a special case of a list with attributes for rows and columns.

Hierarchy: Vectors -> Lists (generalized structure) -> Matrices (special case of vectors) -> Data Frames (special case of lists).

2. What are *attributes*? How can you modify them?

Attributes: Metadata associated with R objects. Common attributes include names, dim (dimensions), dimnames, and class

Modify Attributes:

- Use the attributes() function to view or set all attributes.
- Use specific functions like names() or dim() to access or modify individual attributes.

3. What is special about the attribute called "class"?

- The class attribute determines how R interprets and processes the object.
- Functions like print(), summary(), and arithmetic operations behave differently based on the object's class.

- Modifying the class can change the behavior of an object dramatically.

4. What is dangerous about `stringsAsFactors`? (Hint: What is the *type* for factors?)

What Is `stringsAsFactors`?

- A parameter in functions like `read.table()` or `data.frame()` that, if set to `TRUE`, automatically converts character columns into factors.

Type of Factors:

- Factors are stored as integer codes with associated levels (character labels). The actual type is integer.

Why Is It Dangerous?

- Automatic conversion can lead to unexpected behavior:
 - Factors might not behave as intended in numeric operations since they are integers.
 - Loss of original character data when levels are modified or removed.
 - Issues when combining or merging data frames if levels don't align.

Chapter 4: Subsetting

1. What are the six ways of subsetting vectors?

By Positive Integers:

- Use numeric indices to specify which elements to extract.

```
x <- c("a", "b", "c", "d")
```

```
x[1] # "a"
```

By Character Vectors:

- Use names to extract elements if the vector has a names attribute.

```
x <- c(a = 1, b = 2, c = 3)
```

```
x["b"] # 2
```

By Logical Vector:

- Use a logical vector to select elements where the condition is TRUE.

```
x <- 1:10
```

```
x[x > 5] # 6, 7, 8, 9, 10
```

By Negative integers:

- Use negative indices to exclude elements.

```
x <- c("a", "b", "c", "d")
```

```
x[-2] # "a", "c", "d"
```

By Zero:

- Using 0 removes all elements (returns an empty vector)., good for test data

```
x <- c("a", "b", "c")
```

```
x[0] # character(0)
```

With Nothing [] :

- Provides everything in the og vector,

2. When should you use `drop = FALSE`?

- a. Prevents R from simplifying the result of subsetting a matrix or a data frame into a lower-dimensional structure
- b. To use when you need to preserve the original dimensions
- c. `mat <- matrix(1:9, nrow = 3)`

`mat[1, , drop = FALSE]` # Returns a 1x3 matrix

`mat[1, ]` # Returns a vector

3. What is *recycling* and in what circumstances does it occur?

- a. Recycling happens when the length of one vector is longer than the other, during operations or subsetting

- i. Essentially shorter vector is repeated (i.e. recycled) to fill in the missing length

- ii. `x <- c(2.1, 4.2, 3.3, 5.4)`

`x[c(TRUE, FALSE)]` # Recycled to `c(TRUE, FALSE, TRUE, FALSE)`

Returns: `c(2.1, 3.3)`

4. What are the functions `pluck()` and `chuck()` useful for?

- a. **`pluck()`**: Extracts an element from a list or nested structure safely (returns `NULL` if the element does not exist).
- b. **`chuck()`**: Similar to `pluck()`, but throws an error if the element is missing.
- c. both part of the `purrr` package and are useful for list manipulation

5. What is Boolean masking (aka Boolean algebra) and when is it useful? When might it be overkill?

- a. refers to using logical vectors to subset elements based on conditions

- i. Useful when:

- 1. When filtering data based on specific conditions.
 - 2. concise, expressive filtering logic.

- ii. Overkill when

- 1. For simple selections (e.g., `x[1:2]` is clearer than using a logical vector for small subsets).
 - 2. When the logical vector is dynamically generated but adds unnecessary complexity.

Chapter 5: Control Flow

1. What is the difference between if, else, else if, ifelse, case_when, and switch?
When might you use each?

if: Basic conditional statement to evaluate one logical condition. Executes a block of code if the condition is TRUE, and optionally another if FALSE.

- Use case: When a single condition or a small set of conditions needs to be evaluated.

else: Paired with if to execute code if the if condition is FALSE.

else if: Extends if for multiple conditions, evaluated sequentially until one is TRUE.

- Use case: To handle multiple mutually exclusive conditions.

ifelse: Vectorized version of if, designed for element-wise condition evaluation.

- Use case: Efficiently handle vectors with logical conditions.

case_when: Provided by the dplyr package, allows handling multiple conditions in a concise way with readable syntax.

- Use case: Vectorized conditional logic with multiple conditions and outputs.

switch: Selects one of several options based on a key value. Can use numeric or character input.

2. What happens if you give an if statement a vector of TRUE and FALSE?

If only evaluates the first element:

Warning: the condition has length > 1 and only the first element will be used

3. What is a situation where a while loop might be more useful than a for loop?

While loops are better suited for situations where the number of iterations isn't predetermined and depends on a condition being met

- For example, sampling data until a specific criterion is satisfied.

4. What is the difference between next, break, and print inside a loop?

next: Skips the current iteration and moves to the next.

break: Exits the loop entirely.

print: Outputs values but doesn't control loop flow; useful for debugging.

5. What are good alternatives to using a for loop? When might these not be sufficient or desirable?

Vectorized Functions: `apply`, `lapply`, `sapply`, and `purrr::map` are functional programming constructs for replacing loops in many data processing tasks.

Example: Summing across rows of a matrix.

When Loops Are Needed: If iteration involves complex side effects, conditionally terminated loops (`while`), or performance issues with alternative constructs.

Chapter 6: Functions

1. What are the formals, body, and environment of a function?

Formals: The list of arguments a function accepts, which controls how the function is called. These are specified explicitly when the function is created.

```
f <- function(x, y) x + y
formals(f)
# Output: $x, $y
```

Body: The code block inside the function that is executed when the function is called. It defines the operations performed.

```
body(f)
# Output: { x + y }
```

Environment: The context or data structure that determines how the function resolves variable names. It is implicitly defined by where the function is created.

```
environment(f)
# Output: <environment: R_GlobalEnv>
```

2. What is a primitive function?

Primitive functions are low-level functions in R that directly call C code. They are faster but lack the usual components of other R functions.

Characteristics:

- exist in the **base** package.
- Their `formals()`, `body()`, and `environment()` are `NULL`.
- They are of type "builtin" or "special".

```
typeof(sum)
# Output: "builtin"
formals(sum)
# Output: NULL
```

3. What is lexical scoping? What does this mean about using the same names for objects inside and outside of functions?

Lexical scoping in R means that the value of a variable is determined by its **location in the code** (where it was defined) rather than the runtime call

stack. R looks for variable definitions in the environment where the function was **created**, not where it is **called**.

How Lexical Scoping Works

When a function is executed, R follows these steps to find the value of a variable:

- a. **Search within the function's environment** (local scope).
- b. **Search in the environment where the function was defined** (enclosing scope).
- c. **Search in the parent environments**, all the way up to the global environment.
- d. **If not found, throw an error** (e.g., "object not found")

4. What do we mean by R having "first-class functions"? Why is this useful?

First-class functions mean that functions in R are treated as regular objects.

You can:

- Create functions without assigning names (anonymous functions).
- Store functions in variables, lists, or pass them as arguments.
- Return functions as results of other functions.

Usefulness:

- Promotes functional programming, enabling concise, reusable, and composable code.
- Encourages abstraction and simplifies operations like mapping or filtering.

5. If you pass in an R code expression as an argument to a function, when does the code get evaluated?

If an argument is passed as a code expression (e.g., `x + y`) rather than a computed value, the argument is evaluated lazily when it is actually needed inside the function body

6. How is `&` different from `&&`?

`&` performs **element-wise comparison** of logical vectors.

It processes all elements of the input vectors and returns a logical vector of the same length.

`c(TRUE, FALSE, TRUE) & c(FALSE, TRUE, TRUE)`

```
# Output: FALSE FALSE TRUE
```

&& evaluates **only the first element** of each logical vector.

It is used mainly for control flow where only the first logical value matters.
`c(TRUE, FALSE, TRUE) && c(FALSE, TRUE, TRUE)`

```
# Output: FALSE
```

Key takeaway: Use **&&** when you only care about the first element (e.g., in if conditions) and **&** for element-wise operations.

7. What is "passing the dots" and why might it be useful?

"Passing the dots" refers to using the `...` argument in R functions to pass an arbitrary number of arguments to another function.

Why it is useful:

- **Flexibility:** Allows functions to accept additional arguments without explicitly listing them.
- **Delegation:** Helps forward arguments to other functions seamlessly.
- **Reduces redundancy:** You don't need to redefine all possible arguments when wrapping functions.

8. What are the four types of functions, and how are they different?

1. Primitive Functions:

Built-in functions written in C and called directly by R.

They have no formal arguments or body in the R language.

Example: `sum`, `mean`, `abs`

Checking a primitive function:
`is.primitive(sum) # TRUE`

2. Internal Functions:

Also written in C, but they have an R interface.

Example: `lm` (linear model)

Internal functions can be more complex than primitive functions and include R-level formal arguments.

3. User-defined Functions:

Functions you write yourself in R.

```
my_function <- function(x, y) {  
  
  return(x + y)  
  
}
```

4. Special Functions:

Functions with unique evaluation rules.

Example: if, for, while, repeat, and function

These aren't technically standard functions but instead part of R's control structures.

Summary:

Primitive functions are low-level and fast.

Internal functions bridge C and R code with a formal interface.

User-defined functions provide customization.

Special functions enable control flow and structure in R programs.

Chapter 7: Environments

1. An environment is essentially a list of objects. What is special about an environment that makes it more than just a list?
 - it is a data structure that links variable names to their values in a particular scope or context. The key aspect of an environment is the ability to "look up" the values associated with variable names. Environments also support **lexical scoping**, meaning they contain information about the context in which variables were defined. Environments have a hierarchical structure, where an environment can "enclose" another, and the lookup process can search through this hierarchy to find variable bindings.
2. What is *dynamic scoping*? What consequences does this have for creating functions?
 - rule for variable lookup in which the values of variables are determined by the most recent bindings in the call stack at the time of evaluation, rather than by the place where the function was defined (which is what lexical scoping uses). This means that a function call will resolve variable values based on the environment in which the function is called, not the environment in which it was defined.

Consequences for creating functions:

- , a function might unexpectedly use a variable defined elsewhere in the program if that variable is in scope at the time of function execution.
 - can lead to unintended side effects or bugs because the environment is not strictly controlled by the function's lexical context. This makes reasoning about the behavior of functions more challenging and less predictable.
 - might need more care in managing the environments they operate in, potentially leading to more complex designs or the use of more local variables to avoid interference.
3. What is the difference between a *package environment* and a *namespace environment*? Why is it important that these are different?

A **package environment** is a specific environment associated with a package that stores the objects, functions, and variables that belong to that package. A **namespace environment**, on the other hand, is a specific type of environment that manages the visibility and scope of functions and objects within a package or module.

- **Package environment** refers to the environment where the package's functions and variables are stored.
- **Namespace environment** controls the visibility of functions or variables, ensuring that different parts of a program don't accidentally clash due to naming conflicts. It ensures that functions and variables are only accessible in the intended context.

Importance:

The separation ensures modularity in programming by preventing the accidental overwriting of variables and functions when different packages or libraries are used in the same program.

It allows for better management of package contents, making it easier to update and maintain large codebases where multiple packages or namespaces may be in use.

4. What is the difference between a *function environment* and an *execution environment*? Why is it important that these are different?

function environment: is the environment where a function is defined, containing the variables and bindings that exist when the function is created. It is typically tied to the lexical scope, meaning the function will refer to variables that are available at the time of its definition.

execution environment: is the environment in which a function is actually called or executed, containing the active bindings of variables at that point in time. It reflects the actual state of the program at the time of execution.

Importance of the difference:

- The distinction is important because it allows for **lexical scoping**, where a function remembers its environment (the environment in which it was defined) even when executed in a different context. This supports closure properties in functional programming.
- Without this, function execution could lead to variable name clashes or unexpected behavior
 - function could erroneously reference variables from the execution environment instead of its original function environment.

Chapter 8: Conditions

8.1 Introduction

Condition system: provides a pair of paired set of tools that allow the author of a function to indicate that something unusual is happening

Key tools:

- **Signaling conditions:** `stop()`, `warning()`, `message()`.
- **Handling conditions:** `tryCatch()`, `withCallingHandlers()`.

8.2 Signaling Conditions

Types of Conditions

1. **Errors:** Severe conditions; halt execution.
 - a. Signaled with `stop()` or `rlang::abort()`.
2. **Warnings:** Indicate recoverable issues.
 - a. Signaled with `warning()` or `rlang::warn()`.
3. **Messages:** Provide information; non-critical.
 - a. Signaled with `message()`.

Guidelines for Writing Conditions

- Write clear, helpful messages.
- Use warnings and messages judiciously to avoid overwhelming users.
- Include recovery suggestions in error messages when possible.

8.3 Ignoring Conditions

Functions to ignore conditions (execution continues despite error):

- **Errors:** `try()`.
- **Warnings:** `suppressWarnings()`.
- **Messages:** `suppressMessages()`.

8.4 Handling Conditions

Handlers allow you to override or supplement default behavior for conditions:

- **`tryCatch()`:** Defines *exiting* handlers (for errors and interrupts).
- **`withCallingHandlers()`:** Defines *calling* handlers (for warnings and messages).

8.5 Custom Conditions

- **Why?**
 - Built-in conditions (error, warning, message) often lack metadata, making specific error handling challenging.
 - Custom conditions allow metadata (e.g., problematic arguments, expected types), ensuring better programmability and maintainability.
- **How?**
 - Use `rlang::abort()` to define custom error conditions with `.subclass` and metadata.

Improving Errors with Metadata

1. **Problem:** `log()` error messages are ambiguous.
2. **Solution:** Create custom errors specifying the argument and its invalid type

8.6 Applications

Default Values for Failures:

- Return a fallback value when an error occurs.

```
fail_with <- function(expr, value = NULL) {
  tryCatch(expr, error = function(cnd) value)
}
```

Capturing Conditions:

- Collect all message, warning, and error conditions in a list for later inspection.

```
catch_cnds <- function(expr) {
  conds <- list()
  withCallingHandlers(
    message = function(cnd) conds <- append(conds, list(cnd)),
    warning = function(cnd) conds <- append(conds, list(cnd)),
    tryCatch(expr, error = function(cnd) conds <- append(conds, list(cnd)))
  )
  conds
}
```

Custom Logging:

- Signal custom conditions for logging purposes without default effects.

```
log <- function(message, level = "info") {
```

```
signalCondition(structure(list(message = message, level = level), class = "log"))
}
```

Chapter 8: Conditions

1. Give an example of when you might use an error vs a warning vs a message when designing a function.

Error: Use an error (`stop()`) when the function encounters a condition that prevents it from proceeding further. Errors indicate a failure that the user must address before the function can run correctly.

Example: A function that calculates the square root of a number:

```
sqrt_safe <- function(x) {
  if (!is.numeric(x) || x < 0) {
    stop("Input must be a non-negative number.")
  }
  sqrt(x)
}
```

Warning: Use a warning (`warning()`) when the function encounters a problem but can still proceed by making an assumption or a substitution. Warnings alert the user about potential issues without halting execution.

Example: A function that replaces negative numbers with zero:

```
replace_negatives <- function(x) {
  if (any(x < 0)) {
    warning("Negative values detected; replacing them with 0.")
    x[x < 0] <- 0
  }
  x
}
```

Message: Use a message (`message()`) to inform the user about non-critical information or updates about the function's execution. Messages are purely informational and do not indicate problems.

Example: A function that calculates the mean and reports the number of NA values removed:

```
mean_with_message <- function(x) {
  n_na <- sum(is.na(x))
  if (n_na > 0) {
    message(n_na, " NA values were removed.")
  }
}
```

```
    mean(x, na.rm = TRUE)  
  }
```

2. Why might you want to use `try()` inside your function? When might you prefer to use `tryCatch()` or `withCallingHandlers()` instead?
- useful when you want to handle errors during the execution of a block of code without stopping the entire function
 - allows the function to proceed even if an error occurs in part of the code.
 - `tryCatch()`**: for more sophisticated error handling where you want to define specific behaviors for different types of conditions (errors, warnings, messages).
 - `withCallingHandlers()`**: when you want to handle a condition while still propagating it up the call stack, allows you to modify or suppress warnings/messages without stopping execution or silencing them completely.

Chapter 9: Functionals: Examples to Explore

[Here is a link to the GitHub site for the purrr package.](#)

1. Can you find the source file for the "-ly" functions. What is the purpose of these, and why might they be useful?
 - <https://github.com/tidyverse/purrr/blob/main/R/adverb-safely.R>
 - are designed to modify existing functions to handle errors and side effects more gracefully. These are termed "adverbs" because they modify the behavior of verbs (functions). For example, safely() wraps a function to capture errors, returning a list with components result and error. If the function succeeds, result contains the returned value, and error is NULL. If an error occurs, error contains the error object, and result is either NULL or a specified default value
2. Find the source code for `map\_()` function. What is the purpose of this function?
 - The map() function in purrr applies a function to each element of a list or atomic vector and returns a list. The source code for map() is available on GitHub.
 - allows for non-st enabling you to pass expressions that are evaluated in the context of the data. This is useful when you want to apply a function to each element of a list or vector, but the function involves non-standard evaluation.

[Here is a great blog post about Split-Apply-Combine and Map-Reduce](#)

Questions:

3. What is the main difference between an apply/map process and a reduce process?

Apply/Map (Vectorized Operations):

- Focus: Applies a function to **each element** of a vector, list, or data frame.
- Output: Produces a new collection with transformed elements.
- Functions: lapply(), sapply(), apply(), purrr::map().

```
numbers <- c(1, 2, 3)
```

```
squares <- sapply(numbers, function(x) x^2)
```

```
print(squares) # Output: [1] 1 4 9
```

Reduce:

- Focus: Combines all elements of a collection into a **single value** by iteratively applying a function.
- Output: A single cumulative result.
- Functions: Reduce(), purrr::reduce().

```
numbers <- c(1, 2, 3)
```

```
product <- Reduce(`*`, numbers)
```

- print(product) # Output: 6

4. What are some options when you want to use a functional, but no named function exists to input to the functional that does exactly what you need?

Options for Using Functional Programming When No Named Function Exists in R

1. Anonymous Functions (Inline):

- Use function(x) directly within apply() or similar.

```
numbers <- c(1, 2, 3)
```

```
doubled <- sapply(numbers, function(x) x * 2)
```

```
print(doubled) # Output: [1] 2 4 6
```

2. Define a Small Function:

- Define a custom function with function() and then pass it to the functional.

```
square_plus_one <- function(x) {  
  return(x^2 + 1)  
}
```

```
numbers <- c(1, 2, 3)
```

```
results <- sapply(numbers, square_plus_one)
```

```
print(results) # Output: [1] 2 5 10
```

3. Vectorized Operations (Alternative to Map):

- Use vectorized operations in R for element-wise computation.

```
numbers <- c(1, 2, 3)
```

```
squared <- numbers^2
```

```
print(squared) # Output: [1] 1 4 9
```

○

4. **Chaining with purrr:**

- Use purrr for advanced functional programming.

```
library(purrr)
numbers <- c(1, 2, 3, 4)
total <- numbers %>%
  map(~ .x^2) %>%
  keep(~ .x > 4) %>%
  reduce(`+`)
print(total) # Output: 25
```

5. **Reduce with Base R:**

- Combine elements using a custom function.

```
numbers <- c(1, 2, 3, 4)
total_sum <- Reduce(function(x, y) x + y, numbers)
print(total_sum) # Output: 10
```

6. **Using apply() on Matrices or Data Frames:**

- Process rows or columns with apply().

```
mat <- matrix(1:9, nrow = 3)
row_sums <- apply(mat, 1, sum) # Sum of each row
print(row_sums) # Output: [1] 12 15 18
```

7. **Custom Filters with Filter():**

- Use base R Filter() to preprocess data.

```
numbers <- c(1, 2, 3, 4)
even_numbers <- Filter(function(x) x %% 2 == 0, numbers)
print(even_numbers) # Output: [1] 2 4
```

Chapter 10: Function Factories

- Activity: Create your own example of a "must be forced" situation like in 10.2.3.
 - arises when a function creates a delayed computation that only evaluates under specific circumstances, such as when explicitly called or used in another expression

```
create_counter <- function() {  
  count <- 0  
  function() {  
    count <- count + 1  
    count  
  }  
}  
  
# Example usage  
counter <- create_counter()  
lazy_result <- quote(counter()) # Delayed computation  
print(lazy_result)             # Prints the expression, not the value  
eval(lazy_result)
```

- Discussion: Exercises 2,3,5,6

Base R function factories: `approxfun()` and `ecdf()`

- **`approxfun()`**: Creates a function that linearly interpolates between given x and y values.

```
x <- c(1, 2, 3)  
y <- c(2, 4, 6)  
f <- approxfun(x, y)  
f(2.5) # Returns 5
```

- **`ecdf()`**: Creates a function representing the empirical cumulative distribution function of a numeric vector.

```
data <- c(1, 2, 3, 4)  
f <- ecdf(data)  
  
f(2.5) # Returns 0.5 (50% of values are  $\leq 2.5$ )
```

pick() function

```
pick <- function(i) {  
  function(x) x[[i]]  
}  
  
# Example usage  
x <- list("a", "b", "c")  
pick(2)(x) # Returns "b"  
  
lapply(mtcars, pick(5))  
# Equivalent to  
lapply(mtcars, function(x) x[[5]])
```

Exercise 5:

Without a closure, the `i` variable is modified in the global environment. Successive calls to `new_counter2()` update the same global `i`.

Exercise 6: Using `<-` creates a new local variable `i` inside the returned function's environment every time it's called, so the value of `i` doesn't persist between calls. To maintain state across calls, `<<-` must be used to modify `i` in the enclosing environment.

#Scavenger hunt: Find the source code in `ggplot` where `binwidth` is used as a function. This is hard! Try it for 5-10 minutes, then ping me for a hint.

- Why are function factories particularly useful for random processes, like bootstrapping?
 - helpful for random processes like bootstrapping because they encapsulate the logic for generating random samples or performing resampling into reusable functions.
 - modular design improves clarity and reduces code duplication.
 - in bootstrapping, a function factory can generate functions tailored to specific datasets or resampling parameters, allowing you to apply consistent logic across multiple resampling scenarios
- Discussion: Talk through the design of 10.5.
 - combines function factories (to create reusable, parameterized functions) and functionals (like `purrr::map`) to systematically apply the factory logic to a set of inputs.
 - Function factories encapsulate specific computation logic.
 - Functionals iterate over inputs to create multiple customized functions efficiently.
 - Improves scalability and maintainability by separating function generation (via the factory) from application (via the functional).

Chapter 11: Function Operators

- Discussion: Vectorize() function
 - used to apply a function over a vector or array, element by element, without needing to explicitly loop over the elements
 - particularly useful for optimizing operations on vectors or matrices
- Activity: (Ex 4) Write a function operator that logs a timestamp and message to a file every time a function is run.

```
log_function <- function(func, log_file = "function_log.txt") {  
  function(...) {  
    timestamp <- Sys.time()  
    message <- paste("Function executed at", timestamp)  
    cat(message, "\n", file = log_file, append = TRUE)  
    result <- func(...)  
    return(result)  
  }  
}
```

Example usage:

```
sample_function <- function(x) {  
  return(x^2)  
}
```

Not sure if this was expected end outcome??? check

```
logged_function <- log_function(sample_function)  
result <- logged_function(5)  
print(result) # 25
```

Chapter 12: Base Types

12.1 Introduction

- object-oriented approach (like S3 or S4) introduces additional complexity but also flexibility in how objects and methods are structured.

12.2 Base vs OO Objects

- **Base Objects:**
 - refer to the traditional objects in R that don't adhere to object-oriented programming (OOP) principles. These objects are created and manipulated through basic R functions.
- **OO Objects**
 - refer to objects that adhere to object-oriented systems like S3 or S4, and they have a special class attribute that defines their behavior.
- **Identifying OO Objects:**
 - You can distinguish between base and OO objects using functions like `is.object()` and `sloop::otype()`. For instance:
 - `is.object(1:10)` returns FALSE (base object).
 - `is.object(mtcars)` returns TRUE (S3 object).

12.3 Base Types

Base Types are the fundamental types that define the structure of all objects in R. They include:

- **Vectors:** Types like NULL, logical, integer, double, complex, character, list, and raw.
- **Functions:** Types like closure (for regular R functions), special (for internal functions), and builtin (for primitive functions).
- **Environments:** For example, the global environment is of type environment.
- **S4 Type:**
- **Language Components:** Types like symbol, language, and pairlist are used for language objects in R.
- **Numeric Type:**
 - Numeric Type in R: The term "numeric" is used in different contexts:
 - As an alias for double (e.g., `as.numeric()` is equivalent to `as.double()`).
 - To refer to either integer or double in the S3 and S4 systems when selecting methods.
 - The function `is.numeric()` checks whether an object behaves like a number, but be aware that objects like factors (which have type "integer") won't return TRUE for `is.numeric()` because they don't behave as numbers.

Chapter 13: S3

13.2 Basics

S3 objects: S3 objects are created by setting a class attribute. These objects are built on a base type (such as an integer vector) and may have other attributes.

Generics: S3 uses generic functions that change behavior depending on the class of the object passed to them.

- `print()` is a generic, and when used on a factor object, it calls `print.factor()`.

Methods: Methods are specific implementations of generics for different classes. Methods have a naming convention of `generic.class()`. You should call the generic, not the method directly, to ensure proper method dispatch.

Method Dispatch: The `s3_dispatch()` function from the `sloop` package is used to explore how S3 handles method dispatch, helping identify which method is called for a specific class.

The `as.data.frame.data.frame()` method: can be confusing b/c it may look like a normal method call, but S3 method that behaves differently from others.

- it could return a data frame based on the method's logic.

13.3 Classes

No formal class definition: S3 doesn't have a formal class structure like other languages (e.g., Java).

- can assign any class to an object using `structure()` or `class<-()`.

Creating a new class: When creating an S3 class, you can define a constructor (`new_myclass()`)

- helps create objects with the correct structure.

Validator functions: For more complex classes (like `factor`), it's better to separate checks into a validation function.

- keeps the constructor light and ensures that objects are valid.

Helper functions: If users need to create objects from a class, provide a helper function (`myclass()`), which calls the constructor and validator.

13.4 Generics and Methods

generic function uses `UseMethod()` to dispatch method calls to specific methods based on the class of the first argument.

The method is determined by checking a vector of potential methods, such as `generic.class` and `generic.default`.

Example: The function `mean()` is a generic, which dispatches to methods like `mean.Date` or `mean.default`.

- should only create a method for a generic if you own the generic or class, and the method should match the arguments of the generic.

13.6 Inheritance

- **S3 classes can inherit behaviors from other classes** using a class vector, where a subclass can appear first in the class vector (e.g., ordered is a subclass of factor).
- **NextMethod()** is used to delegate to the next method in the hierarchy, enabling subclasses to modify behaviors while retaining the functionality of the superclass.

When creating a class:

- must decide whether to allow subclasses, which requires modifying the constructor and methods accordingly.

13.5 Object Styles and 13.7 Dispatch details

- **Vector style objects:** These are simple objects where `length(x)` represents the number of observations. Examples include Date and factor classes, which are typically used for one-dimensional data structures.
 - **Dispatch:** based on the class of the object itself, with methods like `mean()` dispatching according to the class of the vector.
- **Record style objects:** Represented by lists of equal-length vectors, these objects like POSIXlt and data frames allow for more complex structures.
 - **Dispatch:** similar, but methods are designed to interact with components of the list. For instance, `length()` and subsetting methods are overridden to conceal the list-based implementation.
- **Scalar objects:** Typically represented by a list, scalar objects like the result of `lm()` represent a single thing (e.g., a model).
 - **Dispatch:** involves treating these objects as singular entities rather than collections, and methods may involve internal dispatch for more specific functionality.

Chapter 17: Big Picture

Metaprogramming: Metaprogramming in R involves treating code as data, allowing you to modify, inspect, and create new code programmatically.

17.2 Code is Data

- Code can be captured and treated as data
- Use `expr()` to capture code: returns the exact code input
- `expr()` is for code typed directly; `enexpr()` is for capturing arguments in functions
- Captured code is called an expression (types: call, symbol, constant, pairlist)
- Modify expressions like lists with `[[` and `$`

17.3 Code is a Tree

- Code is represented as a tree (abstract syntax tree - AST)
- AST shows function calls as branches, symbols/constants as leaves
- `lobstr::ast()` visualizes the AST of code
- Nested function calls create deeper tree structures.
- Useful for understanding R's syntax and grammar.

17.4 Code Can Generate Code

- `call2()` creates function calls programmatically.
- `call2()` is clunky but useful for programming; alternative: unquoting with `!!`.
- `!!` inserts a code tree into an expression.
- Unquoting builds complex code from simpler fragments.
- Creating dynamic expressions allows for flexibility in code generation.

17.5 Evaluation Runs Code

- Evaluation is the process of executing code.
- `eval()` takes an expression and an environment.
- Modifying the environment can tweak code evaluation.
- Custom environments can override functions or create specific evaluation contexts.

17.6 Customising Evaluation with Functions

- You can override operators or functions in a custom environment.
 - Example: custom operators for string manipulation.
- Useful for building domain-specific languages
- Extends functionality by modifying standard evaluation

17.7 Customising Evaluation with Data

- Evaluating code with data frames (data masks) instead of environments.
- `eval_tidy()` evaluates expressions with a data frame context.
- `eval_tidy()` works with data frames
 - enabling simpler syntax (e.g., `x + y` instead of `df$x + df$y`).
- Ambiguity in variable references can arise;
 - `.data` and `.env` pronouns help manage it.

17.8 Quosures

- Problem arises when expressions refer to variables in different environments.
- Quosures bundle expressions with their environment to preserve context.
- Use `enquo()` instead of `enexpr()` when working with data masks.
- `eval_tidy()` works with quosures to evaluate expressions in the correct environment.

Chapter 18: Expressions

18.1 Introduction

- **Concept of Operations vs. Results:** Operation (action) vs. Result (output) of the action.
- **Example Code:** `y <- x * 10` fails due to undefined `x`.
- **Use of `rlang::expr()`:** Captures code structure without executing.
 - Example: `z <- rlang::expr(y <- x * 10)`
 - `eval(z)` executes the captured expression.
- **Chapter Focus:** Data structures behind expressions.
 - `expr()` in Chapter 19, `eval()` in Chapter 20.

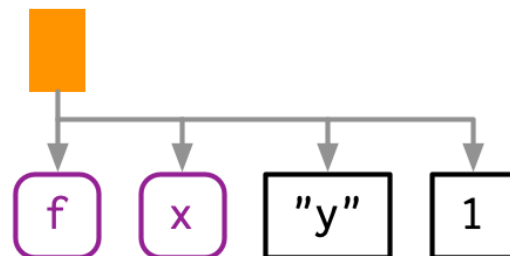
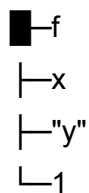
18.2 Abstract Syntax Trees (ASTs)

- **AST Definition:** Hierarchical tree representing R code structure.
- **Purpose:** Understanding the structure for metaprogramming (inspecting and modifying code).

18.2.1 Drawing ASTs

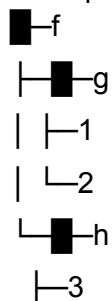
- **Conventions for Drawing:**
 - Use symbols and constants as leaves; function calls as branches.
 - `lobstr::ast()` visualizes trees.

Example tree: `f(x, "y", 1)` becomes:



- **Multi-level Trees:** More complex expressions create deeper trees.

Example: `f(g(1, 2), h(3, 4, i()))`





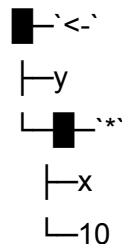
18.2.2 Non-code Components

- **Whitespace and Comments:** AST ignores comments and most whitespace.
 - `f(x, y) # comment` becomes the same tree as `f(x, y)`.
- **Whitespace Effect:** Whitespace can affect operator behavior, as in `y <- x` vs. `y <-x`.

18.2.3 Infix Calls

- **Infix to Prefix:** Example: `y <- x * 10` is equivalent to `<-(y, *(x, 10))`.

AST for `y <- x * 10`:



18.3 Expressions

- **Definition:** Expressions are combinations of constants, symbols, and calls that form the AST.

18.3.1 Constants

- **Scalar Constants:** Simple values like `TRUE`, `1L`, `"x"`.
- **Self-quoting:** `expr(TRUE)` is identical to `TRUE`.

18.3.2 Symbols

- **Symbol Definition:** Names of objects like `x`, `mtcars`.
- **Creating Symbols:** Use `expr(x)` or `sym("x")`.
- **Conversion:** Turn symbols into strings using `as.character()` or `rlang::as_string()`.
- **Symbol Example:** `is.symbol(expr(x))` returns `TRUE`.

18.3.3 Calls

- **Call Objects:** Represent function calls in the AST. First element is the function, others are arguments.
- **Identifying Calls:** Use `is.call()` to check for call objects.
- **Subsetting Calls:** Extract function and arguments using `[[1]]`, `$`, or `length(x) - 1`.
 - Example: `x[[1]]` returns the function name.

18.3.3.1 Subsetting

- Extracting arguments via `[]` or `$`.
- Example: `x$row.names` extracts an argument by name.

18.3.3.2 Function Position

- **First Element:** The function being called.
- **Symbol Example:** `lobstr::ast(foo())` shows the function `foo` as the root.

18.3.3.3 Constructing Calls

- **Creating Calls:** Use `rlang::call2()` to create calls.
 - Example: `call2("mean", x = expr(x), na.rm = TRUE)`.

18.3.4 Summary

- Scalar constants: `logi/int/num/chr` types.
- Symbols: `symbol` type.
- Call objects: `language` type.
- Pairs and expression vectors are specialized cases.

18.4 Parsing and Grammar

18.4.1 Operator Precedence

Infix functions can introduce ambiguity in expressions. For example, when evaluating `1 + 2 * 3`, the question arises: should it be evaluated as `(1 + 2) * 3` or `1 + (2 * 3)`?

Programming languages use **operator precedence** to resolve this ambiguity.

To check how R resolves this, we can use `lobstr::ast()`:

```
lobstr::ast(1 + 2 * 3)
```

```
#> ──`+`─
```

```
#> ──1─
```

```
#> ──┬─`*`─
```

```
#> ──2─
```

```
#> ──3─
```

R uses **multiplication (*)** with higher precedence than addition (+), so `1 + (2 * 3)` evaluates to 7. In other cases, like the operator `!`, precedence might be surprising:

```
lobstr::ast(!x %in% y)
```

```
#> ──`!`
```

```
#> └─┬─`%in%`
```

```
#> │ └─x
```

```
#> │ └─y
```

R uses a variety of precedence groups for infix operators. It's best to use parentheses if you're unsure about precedence.

```
lobstr::ast((1 + 2) * 3)
```

```
#> ──`*`
```

```
#> └─┬─`(`
```

```
#> │ └─┬─`+`
```

```
#> │ │ └─1
```

```
#> │ │ └─2
```

```
#> │ └─3
```

18.4.2 Associativity

A second ambiguity arises when an infix operator is used repeatedly. For example, $1 + 2 + 3$ could be parsed as either $(1 + 2) + 3$ or $1 + (2 + 3)$. This is known as **associativity**.

Most operators in R are **left-associative**, meaning operations on the left are evaluated first:

```
lobstr::ast(1 + 2 + 3)
```

```
#> ──`+`
```

```
#> └─┬─`+`
```

```
#> │ └─1
```

```
#> │ └─2
```

```
#> └─3
```

However, there are exceptions like **exponentiation** (^) and **assignment** (<-), which are **right-associative**:

```
lobstr::ast(2^2^3)
```

```
#> └─`^`
```

```
#> └─2
```

```
#> └─└─`^`
```

```
#> └─2
```

```
#> └─3
```

For assignment, this means you can chain assignments like `x <- y <- z <- 0`:

```
lobstr::ast(x <- y <- z)
```

```
#> └─`<`
```

```
#> └─x
```

```
#> └─└─`<`
```

```
#> └─y
```

```
#> └─z
```

18.4.3 Parsing and Deparsing

When working with strings of code, you can **parse** and **deparse** them into expressions.

1. **Parsing:** Use `rlang::parse_expr()` to convert a string into an expression:

```
x1 <- "y <- x + 10"
```

```
x2 <- rlang::parse_expr(x1)
```

```
is.call(x2) # TRUE
```

For multiple expressions, use `rlang::parse_exprs()`:

```
x3 <- "a <- 1; a + 1"

rlang::parse_exprs(x3)

#> [[1]]
#> a <- 1
#>
#> [[2]]
#> a + 1
```

2. **Deparsing:** To convert an expression back to a string, use `rlang::expr_text()`:

```
z <- expr(y <- x + 10)

expr_text(z)

#> [1] "y <- x + 10"
```

Important: `deparse()` can return a vector when the input has multiple lines. `expr_text()` avoids this by returning a single string:

```
deparse(expr)

#> [1] "g(a + b + c + d + e + f + g + h + i + j + k + l + m + n + o + p + q + r + s + t + u + v + w + x + y + z)"

expr_text(expr)

#> [1] "g(a + b + c + d + e + f + g + h + i + j + k + l + m + n + o + p + q + r + s + t + u + v + w + x + y + z)"
```

18.5 Walking AST with Recursive Functions

18.5.1 Finding F and T

We can use recursion to walk through the AST and detect logical abbreviations T and F.

1. **Base case:** Check if the expression is a **constant** or a **symbol**.
2. **Recursive case:** For calls and pairlists, recursively check subcomponents.

```
logical_abbr_rec <- function(x) {  
  switch_expr(x,  
    constant = FALSE,  
    symbol = as_string(x) %in% c("F", "T"),  
    call = purrr::some(x, logical_abbr_rec),  
    pairlist = purrr::some(x, logical_abbr_rec)  
  )  
}
```

```
logical_abbr(mean(x, na.rm = T))
```

```
#> [1] TRUE
```

18.5.2 Finding All Variables Created by Assignment

You can recursively identify all variables created by assignment using `find_assign_rec()`. For each assignment, check if it's a call to `<-`, and return the variable name.

```
find_assign_rec <- function(x) {  
  switch_expr(x,  
    constant = ,  
    symbol = character(),  
    pairlist = flat_map_chr(as.list(x), find_assign_rec),  
    call = {  
      if (is_call(x, "<-")) {
```

```
    as_string(x[[2]])  
  } else {  
    flat_map_chr(as.list(x), find_assign_rec)  
  }  
}  
)  
}
```

Chapter 19: Quasiquotation

19.1 Introduction

- **Quasiquotation** is the process of capturing and selectively evaluating parts of code.
- It is used in tidy evaluation to allow users to combine their code with a function's author's code.
- **Quotation** refers to capturing an expression without evaluating it.
- **Unquotation** refers to selectively evaluating parts of a quoted expression, allowing dynamic modifications.
- Quasiquotation makes it easier to create functions that combine fixed code and user-defined expressions, addressing challenging programming problems.
- This is one of the three pillars of tidy evaluation, along with quosures and the data mask.

19.2 Motivation

- Example of a function, **cement()**, which automatically quotes its arguments.
- **Problem:** When variables are used, the function still treats them as quoted, which isn't ideal.
- **Solution:** Use **unquoting (!!)** to evaluate specific variables inside quoted expressions.
- **Example:** `cement(Good, !!time, !!name)` unquotes time and name, resulting in evaluated values.
- The **difference:** `paste()` evaluates all arguments, while `cement()` quotes them by default.
- **Vocabulary:**
 - **Evaluated argument:** follows R's usual evaluation rules.
 - **Quoted argument:** captured by the function, processed in a custom way.

19.3 Quoting

- **Quoting:** Capturing an expression without evaluating it.
- Four main quoting functions in **rlang**:
 - **expr():** Captures the expression exactly as typed.
 - **enexpr():** Captures the expression supplied by the caller (via lazy evaluation).
 - **exprs():** Captures multiple expressions as a list.
 - **enexprs():** Captures multiple arguments as expressions.
- **Base R equivalents:**
 - **quote()** (equivalent to `expr()`),
 - **substitute()** (equivalent to `enexpr()`),
 - **alist()** (equivalent to `exprs()`).

19.4 Unquoting

- **Unquoting** is the ability to evaluate parts of a quoted expression.
- **!!:** Used to unquote a single argument, replacing it with its evaluated value.
- **Example:** `expr(f(!!x, y))` results in `f(-1, y)` if `x` is `-1`.

- Unquoting works with symbols, constants, and function calls:
 - `expr(f(!a, !b))` results in `f(y, 1)` if `a` is a symbol and `b` is a constant.

19.4.1 Unquoting one argument

- `!!` evaluates a single argument in the expression.
- Example: `expr(f(!x, y))` unquotes `x` and keeps `y` as is.
- **Operator precedence:** When unquoting, it ensures proper operator precedence is maintained, e.g., `expr(!x1 / !x2)` results in `(x + 1) / (x + 2)`.

19.4.2 Unquoting a function

- `!!` can also unquote the function itself.
- **Example:** `expr(!f(x, y))` unquotes the function `f(x, y)` and calls it with `x` and `y`.
- This works with functions as well, e.g., `expr(!pkg::foo)(...)`.

19.5 Non-quoting:

- **bquote() and its limitations:**
 - **bquote()** is used for quasiquotation, where `.`() is used for unquoting.

```
xyz <- bquote((x + y + z))
```

```
bquote(-(xyz) / 2)
```

```
# Output: -(x + y + z)/2
```

- Challenges with **bquote()**:
 - difficult to apply to user-supplied code.
 - doesn't support unquote-splicing for multiple expressions stored in a list.
 - can't handle environments needed for functions evaluating code within a data frame context (e.g., `subset()`).
- **Non-quoting techniques:**

Quoting and non-quoting functions: Functions like `$` or `[[` allow indirect specification.

```
x <- list(var = 1, y = 2)
```

```
var <- "y"
```

```
x$var # Output: 1
```

```
x[[var]] # Output: 2
```

- **Argument-based quoting:** Functions like `rm()` allow bare variable names or a character vector for variable names in list.

- **Argument control:** Functions like `library()` use a `character.only` argument to control quoting behavior.
- **Quoting failure:** Functions like `help()` quote arguments if evaluation fails (e.g., when `var` is undefined).

Example using a non-standard evaluation (NSE) function like `plot()`:

```
palette(RColorBrewer::brewer.pal(3, "Set1"))

plot(Sepal.Length ~ Petal.Length, data = iris, col = Species, pch = 20, cex
= 2)
```

19.6 ... (dot-dot-dot):

- **Problems solved with the `!!!` operator:**
 - **Unquote-splicing:** Allows you to insert elements from a list directly into a function call.

Example of row-binding multiple data frames:

```
dfs <- list(a = data.frame(x = 1, y = 2), b = data.frame(x = 3, y = 4))
```

```
dplyr::bind_rows(!!!dfs)
```

Output: Combined data frame with rows from both data frames.

Indirect argument names: Use `!!` to unquote the value of a variable for an argument name.

The operator `:=` is used for dynamic assignment of column names.

19.6.1 `list2()`:

- Used to pass arguments to functions that don't natively support

Example of `set_attr()` to set attributes flexibly:

```
set_attr <- function(.x, ...) {
```

```
  attr <- rlang::list2(...)
```

```
  attributes(.x) <- attr
```

```
  .x
```

```
}
```

Example output:

```
1:10 %>% set_attr(w = 0, !!!attrs, !!attr_name := 3) %>% str()
```

19.6.2 `exec()`:

- **`exec()`** allows functions without tidy dots to accept indirect arguments.

Example:

```
exec("mean", x = 1:10, na.rm = TRUE, trim = 0.1)
```

You can supply arguments from a list:

```
args <- list(x = 1:10, na.rm = TRUE, trim = 0.1)
```

```
exec("mean", !!!args)
```

19.6.3 `dots_list()`:

- **`dots_list()`** is used to collect arguments passed via ... into a list.
 - You can control how empty or duplicate arguments are handled with `.ignore_empty` and `.homonyms`.

Example:

```
dots_list(x = 1, x = 2, .homonyms = "first")
```

Output: List with only the first x.

19.6.4 Base R (`do.call()`):

- **`do.call()`**: A function that calls other functions with arguments specified in a list.

Example for `rbind()`:

```
do.call("rbind", dfs)
```

◦

Handling indirect arguments:

```
args <- list(val)
```

```
names(args) <- var
```

```
do.call("data.frame", args)
```

19.7 Case Studies:

- **19.7.1 `lobstr::ast()`:**

Demonstrates how quasiquotation solves the problem of capturing and printing expressions:

```
z <- expr(foo(x, y))
```

```
lobstr::ast(!!z)
```

- **19.7.2 Map-reduce to generate code:**

Combine quasiquotation and `purrr::map()` to generate code dynamically based on model coefficients.

```
coef_sym <- syms(names(coefs))
```

```
summands <- map2(coef_sym, coefs, ~ expr(!!x * !!y))
```

```
eq <- reduce(summands, ~ expr(!!x + !!y))
```

Other notes:

Don't use quote and substitute

Easy to help everyone write custom functions

Moments where !! doesn't work

Within mutate, cannot use a symbol within an expression like !! var_name

Chapter 20: Evaluation

20.2 Evaluation Basics:

- **eval():** Evaluates expressions in a given environment.
 - Example: `eval(expr(x + y), env(x = 1000))` evaluates to 1002.
- **Applications:**
 - `local():` Creates temporary variables in a new environment to prevent long-term effects.
 - `source():` Reads and evaluates code from a file.
- **Gotcha:** Functions may have unexpected `srcref` attributes when using `eval()` and `expr()`. Remove them with `attr(f, "srcref") <- NULL` to avoid issues.

20.3 Quosures:

- **Quosures:** Data structure that pairs an expression with its environment.
 - Created using `enquo()`, `enquos()`, or `new_quosure()`.
 - Example: `quo(x + y)` captures the expression `x + y` with its environment.
- **Evaluating Quosures:** Use `eval_tidy()` to evaluate quosures.
 - Example: `eval_tidy(q1)` evaluates a quosure containing `x + y`.
- **Dots and Quosures:** Essential for evaluating arguments passed to functions via `...`, as each argument may have its own environment.

20.4 Data Masks

- **Basics:** Data masks allow referencing variables from an environment and a data frame in a single expression (e.g., `eval_tidy()`).
- **Pronouns:** Use `.data` for data mask variables and `.env` for environment variables to avoid ambiguity.
- **Application: subset():** `subset2()` simplifies row selection using conditions evaluated in the data mask context.
- **Application: transform():** `transform2()` adds variables to data frames by evaluating expressions in the context of the data frame.
- **Application: select():** `select2()` evaluates column selections using `eval_tidy()` to simplify syntax for choosing columns.

20.5 Using Tidy Evaluation

- **Quoting and Unquoting:** Use `enquo()` to capture and `!!` to unquote expressions, enabling dynamic evaluation.
- **Handling Ambiguity:** Use `.data` for the data mask and `.env` for the environment to resolve conflicts.
- **Quoting and Ambiguity:** Carefully unquote and use pronouns to avoid ambiguity when both quoting and evaluating.

20.6 Base Evaluation

- **substitute()**: Base R uses `substitute()` and `eval()` for non-standard evaluation, but it can be more error-prone compared to tidy evaluation approaches like `eval_tidy()`.

Chapter 21: Translating R code

HTML:

- involves capturing an R expression, evaluating it in a special environment, and converting functions and symbols into HTML equivalents. The `to_html()` function helps with this conversion.

LATEX:

Handling Symbols:

- Known symbols like Greek letters (π) are mapped to LaTeX equivalents, while unknown symbols stay the same
- The abstract syntax tree (AST) is walked to identify and convert symbols.

Dealing with Functions:

- Known functions like `sin()` or `log()` are converted into their LaTeX forms
- unknown functions are wrapped in `\textrm{}` or `\mathrm{}` for LaTeX compatibility
 - ensuring all expressions are properly formatted.

Chapter 22: Debugging:

22.2 Overall Approach:

1. Google:
2. Make it repeatable
 - a. Reproducible example, maybe some automated tests
3. Figure Out Where It Is
 - a. Scientific method, systematic approach to the identifying the line of code that is causing the issue.
4. Fix it and Test
 - a. Make sure to record the correct output

22.3 Locating Errors:

- Make an error repeatable
- Show traceback
 - Shows which line

22.3.1 Lazy Evaluation:

- lazy evaluation involved: Section 7.5.2
 - Try using `rlang::with_abort()` and `rlang::last_trace()`, to find source of problem

22.4 Interactive Debugger:

- Sometimes you need more information than just the location of the error, in those cases use `rerun` with `debug`,

22.4.1: Try using `browser()` function call

- `n`: next step
- `s`: works like next except if the next step is a function it'll step into that function for you to analyze
- `f`: finishes execution of current loop or function
- `c`: leaves debugging and continues running through the function
 - Useful if you've fixed the bad state and want to see if everything else works
- `q`: quits in that it stops debugging, terminates the function and returns to workspace
 - Useful when you've figured out the problem and are ready to fix it

22.4.2: Three Alternatives:

Instead of `browser()`: you can

1. set breakpoints in RStudio:
 - a. Shift +F9 or clicking to the left of a line number. Similar to `browser` and easier to set
 - b. Sometimes breakpoint doesn't work and RStudio does not support conditional breakpoints

2. `options(error = recover):`
 - a. For each error you get an interactive prompt that displays the traceback and provides the ability to interactively debug any of the frames
3. `debug()`
 - a. Inserts the browser function for you
 - b. `undebug()` removes it and `debugonce()` browses just the next run
 - c. `utils::setBreakpoint()` works much in the same way, but it instead of function name, takes the file name and line number to find the appropriate function for you.
4. Call Stack
 - a. Call stacks- what are they?
 - b. Call stacks generated by `traceback()`, `browser()` & where `andrecover` are inconsistent

22.4.3 Compiled Code:

- Possible to use an active debugger like `gdb` or `lldb`

22.5 Non-Interactive Debugging:

22.5.1 `dump.frames()`

22.5.2 Print debugging

22.6 Non-Error Failures:

Function may generate an unexpected warning

-

Jenny Bryan: [📺 Jenny Bryan | Object of type 'closure' is not subsettable | RStudio \(2020\)](#)

- Run it all again, turn it off and on again
- Don't keep running the same code
- Make a reprex, like you're showing it to the world.
- `options(error = recover)`
- Certain data structures are particularly prone to bugs
- Closure - > sometimes can call it as function